CSS Variables & Custom Properties

What are CSS Variables?

- CSS Variables (also called **Custom Properties**) are **reusable values** defined once and used throughout the stylesheet.
- They make CSS **cleaner, consistent, and easier to maintain**.

Syntax

1. Defining a variable

```
:root {
  --main-color: #3498db;
  --font-size: 16px;
}
```

- Variables must start with `--`.
- Usually defined in `:root` so they are global.

2. Using a variable

```
h1 {
  color: var(--main-color);
  font-size: var(--font-size);
}
```

Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --primary-bg: lightblue;
      --secondary-bg: coral;
      --text-color: #333;
      --padding-size: 20px;
    }

    body {
      background: var(--primary-bg);
      color: var(--text-color);
      font-family: Arial, sans-serif;
    }

    .box {
      background: var(--secondary-bg);
      padding: var(--padding-size);
      margin: 20px;
      border-radius: 8px;
    }
  </style>
</head>
<body>
  <h1>Welcome to CSS Variables</h1>
  <div class="box">This is a reusable box using CSS variables.</div>
</body>
</html>
```

Variable Scope

- **Global Scope** → Defined inside `:root`, can be used anywhere.
- **Local Scope** → Defined inside a selector, can only be used in that selector.

Example:

```
:root {
  --main-color: blue;   /* Global */
}

.card {
  --main-color: red;    /* Local (only inside .card) */
  color: var(--main-color);
}
```

Fallback Values

You can set a **default value** if the variable is not defined.

```
h1 {
  color: var(--non-existing, black);
}
```

Changing Variables with JavaScript

CSS variables can be updated dynamically using JS.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    :root {
      --main-bg: lightgreen;
    }
    body {
      background: var(--main-bg);
      transition: background 0.5s;
    }
  </style>
</head>
<body>
  <button onclick="changeTheme()">Change Theme</button>

  <script>
    function changeTheme() {
      document.documentElement.style.setProperty('--main-bg',
'lightcoral');
    }
  </script>
</body>
</html>
```

Clicking the button changes the background color.

Advantages of CSS Variables

Easy to **update and maintain**.
Helps in creating **themes** (dark mode/light mode).
Reduces code duplication.
Works well with JavaScript.

CSS Pseudo-classes & Pseudo-elements

A. Pseudo-classes

- A **pseudo-class** defines a special state of an element.
- Written using a **colon (:)**.

Common Pseudo-classes:

1. :hover → When the mouse is over an element.

```
button:hover {
  background: blue;
  color: white;
}
```

2. :active → When an element is clicked.

```
button:active {
  transform: scale(0.95);
}
```

3. **:focus** → When an input field is focused.

```
input:focus {
  border: 2px solid green;
}
```

4. **:first-child / :last-child**

```
p:first-child { color: red; }
p:last-child { color: blue; }
```

5. **:nth-child(n)**

```
li:nth-child(2) { color: orange; } /* 2nd element */
li:nth-child(odd) { background: lightgray; }
```

B. Pseudo-elements

- A **pseudo-element** allows you to style **specific parts of an element**.
- **Written using double colons (::).**

Common Pseudo-elements:

1. **::first-line** → Styles the first line of text.

```
p::first-line {
  font-weight: bold;
  color: red;
}
```

2. **::first-letter** → Styles the first letter.

```
p::first-letter {
  font-size: 30px;
  color: blue;
```

```
}
```

3. **::before** → Insert content **before** an element.

```
h1::before {
  content: "Before";
  color: green;
}
```

4. **::after** → Insert content **after** an element.

```
h1::after {
  content: "After";
  color: orange;
}
```

5. **::selection** → Styles text when highlighted.

```
p::selection {
  background: yellow;
  color: black;
}
```

Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    a:hover {
      color: red;
    }

    p::first-letter {
      font-size: 24px;
      color: blue;
    }

    h2::before {
      content: "* ";
      color: gold;
    }

    h2::after {
      content: " *";
      color: gold;
    }
  </style>
</head>
<body>
  <a href="#">Hover over me</a>
  <p>This is an example of pseudo-elements.</p>
  <h2>Heading</h2>
</body>
</html>
```

Key Points

- **Pseudo-class (:)** → styles element states (hover, active, focus, etc.).
- **Pseudo-element (::)** → styles parts of an element (before, after, first-line, etc.).
- They make styling **interactive and powerful**.

Advanced CSS

CSS Functions

1. **calc()** → Do calculations directly in CSS.

```
.box {
  width: calc(100% - 50px);
}
```

2. **min()** → Chooses the **smallest value**.

```
.box {
  width: min(80%, 600px); /* whichever is smaller */
}
```

3. **max()** → Chooses the **largest value**.

```
.box {
  width: max(300px, 50%); /* whichever is larger */
}
```

4. **clamp()** → Restrict value between min and max.

```
.box {
  font-size: clamp(14px, 2vw, 20px); /* min:14, preferred:2vw, max:20 */
}
```

CSS Filters

Used to apply visual effects to images or elements.

```
img {
  filter: blur(5px) brightness(120%) contrast(150%);
}
```

- **blur(px)** → adds blur.
- **brightness(%)** → adjusts brightness.
- **contrast(%)** → adjusts contrast.
- **grayscale(%)**, **sepia(%)**, **invert(%)**, etc.

Shadows

1. Box Shadow

```
.box {
  width: 150px;
  height: 150px;
  background: lightblue;
  box-shadow: 5px 5px 15px rgba(0,0,0,0.3);
}
```

2. Text Shadow

```
h1 {
  text-shadow: 2px 2px 5px gray;
}
```

Gradients

Gradients are backgrounds that smoothly transition between colors.

1. **Linear Gradient**

```
div {
  background: linear-gradient(to right, red, yellow);
}
```

2. **Radial Gradient**

```
div {
  background: radial-gradient(circle, red, yellow, green);
}
```

3. **Conic Gradient**

```
div {
  background: conic-gradient(red, yellow, green, blue);
}
```

Practical Styling

Navbar Design

```
nav {
  background: #333;
  padding: 10px;
}
nav a {
  color: white;
  text-decoration: none;
  margin: 10px;
}
nav a:hover {
  color: yellow;
}
```

Card Design

```
.card {
  width: 250px;
  padding: 20px;
  background: white;
  border-radius: 10px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.2);
}
```

Buttons & Hover Effects

```
.btn {
  background: blue;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  transition: 0.3s;
}
.btn:hover {
  background: darkblue;
```

```
  transform: scale(1.05);
}
```

Form Styling

```
input, textarea {
  width: 100%;
  padding: 10px;
  margin: 8px 0;
  border: 1px solid #ccc;
  border-radius: 5px;
}
input:focus {
  border-color: blue;
  outline: none;
}
```

Layouts

2-column layout

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}
```

3-column layout

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
```

Header-Footer Layout

```
.container {
  display: grid;
  grid-template-rows: auto 1fr auto;
  height: 100vh;
}
header, footer {
  background: #333;
  color: white;
  padding: 10px;
}
```

Best Practices

Writing Clean CSS

- Use meaningful class names (`.navbar`, `.btn-primary`) instead of `.box1`, `.abc`.
- Keep indentation consistent.

Avoiding Redundancy

Bad:

```
h1 { font-size: 20px; }
```

```
h2 { font-size: 20px; }
```

Good:

```
h1, h2 { font-size: 20px; }
```

Commenting in CSS

```
/* This styles the navigation bar */
nav {
  background: #333;
}
```

External Stylesheet for Reusability

Instead of inline or internal CSS, use an external file:

```
<link rel="stylesheet" href="styles.css">
```